

Week 1 - Day 4b Supplementary: Loop Unrolling and Debugging

Exercise 1: Triangle Types []

Write a function called `check_triangle` that takes three side lengths as input and determines the type of triangle based on the following conditions:

- If all three sides are equal, it is an equilateral triangle.
- If exactly two sides are equal, it is an isosceles triangle.
- If all three sides are different, it is a scalene triangle.

The function should return a `string` indicating the type of triangle. Write the `check_triangle` function that implements these conditions and returns the type of triangle based on the given side lengths. Test your function with different side lengths to verify its correctness.

```
# TODO: Your code here

# Test your code
assert check_triangle(2, 4, 4) == 'isocles'
assert check_triangle(1, 2, 3) == 'scalene'
assert check_triangle(3, 3, 3) == 'equilateral'

print("All tests passed []")
```

Exercise 2: Banking! []

The code below is incomplete. You should complete it so that it performs the tasks mentioned below.

Implement given functions: `deposit()`, `withdrawal()`, and `bank()` so that your program works like an ATM.

Your program should first ask the user "deposit(d) or withdrawal(w) or balance check(c)?" Then:

- If user input is empty string, "", then return nothing.
- If user input is 'w', then ask the amount of money to be withdrawn and withdraw it.
- If user input is 'd', then ask the amount of money to be deposited and deposit it.
- If user input is 'c', then check the current balance.

Hints:

1. We have created a global variable "balance" that will have the value of the money you have in the bank. You need to access this variable every time you need to deposit, withdraw, or check your balance.

2. Normally global variables can be accessed but not changed from inside functions. Use the global keyword to access and change global variables from inside the function.
3. There cannot be a negative balance in the bank.

Begin coding below:

```
# TODO: Complete the code below

balance = 0

def deposit(amount):
    # TODO: YOUR CODE HERE

    # Write code here to add the `amount` to the `balance`.

    print("You have deposited" + str(amount) + "birr")

def withdraw(amount):
    # TODO: YOUR CODE HERE

    # Write code here to withdraw the `amount` from the `balance`.
    # If the amount is larger than the balance, make sure to print a
    message
    # telling the customer that the withdrawal is improper.

    # As an example for how to write print statements,
    # see the print line written above in the deposit function.
    pass

def bank():
    while True:
        process = input("Deposit(d) or withdrawal(w) or balance
check(c)? ")
        if process == "d":
            amount = float(input("How much do you want to deposit?"))
            # TODO: Your code here
        if process == "c":
            print('your current balance is ' + str(balance) + 'birr')
        if process == "w":
            amount = float(input("How much do you want to withdraw?"))
            # TODO: Your code here
        if process == "stop":
            break
        else:
            print("Please type d, w, c, or stop, then press enter.")

bank()
```

Exercise 3: Palindrome []

Given an integer `x`, return `true` if `x` is a palindrome, and `false` otherwise.

Note: A palindrome is a number that reads the same forwards and backwards.

```
# TODO: Your code here
def is_palindrome(x):
    pass

# Test your code
assert is_palindrome(121) == True
assert is_palindrome(126621) == True
assert is_palindrome(42214) == False

print("All tests passed []")
```

Exercise 4: Perfect Numbers []

Write a Python function to check whether a number is perfect or not.

Note: In number theory, a perfect number is a positive integer that is equal to the sum of its proper positive divisors, that is, the sum of its positive divisors excluding the number itself. Equivalently, a perfect number is a number that is half the sum of all of its positive divisors (including itself).

Example : The first perfect number is 6, because 1, 2, and 3 are its proper positive divisors, and $1 + 2 + 3 = 6$. Equivalently, the number 6 is equal to half the sum of all its positive divisors: $(1 + 2 + 3 + 6) / 2 = 6$.

The next perfect number is $28 = 1 + 2 + 4 + 7 + 14$. This is followed by the perfect numbers 496 and 8128.

```
# TODO: Your code here
def is_perfect(n):
    pass

# Test your code
assert is_perfect(6) == True
```